

Secure File Sharing Application: A Hybrid Post-Quantum Approach for Offline Peer-to-Peer File Transfer

Bhuvaneswaran B

*Supervisor, Assistant Professor (SG)
Rajalakshmi Engineering College
Chennai, India
bhuvaneswaran@rajalakshmi.edu.in*

Skandan Kamal

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701322@rajalakshmi.edu.in*

Sidharth Subramonian

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701315@rajalakshmi.edu.in*

Jayan A

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701128@rajalakshmi.edu.in*

Sivaramakrishnan R

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701319@rajalakshmi.edu.in*

Abstract—Modern file-sharing applications commonly depend on internet connectivity, cloud infrastructure, or platform-specific ecosystems, and securely exchanging sensitive files between untrusted devices requires reliable device authentication and strong cryptographic protection. This challenge is more significant in environments where internet access is unavailable or intentionally restricted, such as disaster-response operations, remote locations, and air-gapped networks. This paper presents a Secure File Sharing Application supporting offline peer-to-peer file transfer. The system establishes a peer-to-peer connection using the Nearby Connections API, authenticates devices through out-of-band pairing, performs a hybrid post-quantum key exchange using ML-KEM-768 combined with the classical X25519 algorithm, derives a session key using HKDF-SHA256, and encrypts files using AES-256-GCM. A survey of twenty recent works on hybrid post-quantum cryptography, spanning TLS, 5G, IoT, embedded systems, and formal verification, is used to justify the design choices made. The results demonstrate the feasibility of combining peer-to-peer networking, post-quantum cryptography, and authenticated encryption in a single lightweight mobile application.

Keywords—Post-Quantum Cryptography, ML-KEM, X25519, AES-256-GCM, Hybrid Key Exchange, Peer-to-Peer File Sharing, Nearby Connections API, Threat Model

I. INTRODUCTION

Online file-sharing tools increasingly depend on internet connectivity, cloud infrastructure, or platform-specific ecosystems. While solutions such as AirDrop and other peer-to-peer tools enable local transfers, securely exchanging sensitive files between untrusted devices requires reliable device authentication and strong cryptographic protection [1]. This challenge is amplified in environments where internet access is unavailable, unreliable, or intentionally restricted, such as disaster-response operations, remote locations, hospitals, defense environments, and air-gapped networks. In such settings, a compromised or impersonated device can cause sensitive files to be intercepted or transferred to the wrong recipient.

With the increasing sensitivity of data exchanged between personal and professional devices — medical

records, confidential documents, maps, and images — the risk of interception or misdirected transfer has grown substantially. Existing solutions either rely heavily on cloud infrastructure, exposing data to third-party servers, or lack explicit peer-identity verification, leaving them vulnerable to man-in-the-middle attacks. Furthermore, the eventual arrival of sufficiently capable quantum computers threatens classical key-exchange algorithms such as RSA and ECC, motivating the adoption of hybrid post-quantum primitives even in everyday consumer applications [16].

This paper presents a secure file-sharing system that supports offline peer-to-peer file transfer. The system uses out-of-band pairing verification and a hybrid post-quantum key exchange combining ML-KEM-768 with X25519, followed by HKDF-SHA256 key derivation and AES-256-GCM authenticated encryption, so that files can be exchanged between nearby devices without any internet connectivity or server. The specific contributions of this work are as follows:

- 1) A hybrid, air-gapped, serverless peer-to-peer transfer pipeline combining out-of-band pairing, ML-KEM-768 + X25519 key exchange, HKDF-SHA256 derivation, and AES-256-GCM encryption.
- 2) A threat model and security discussion connecting the design to twenty recent works on post-quantum cryptography.
- 3) A working Android implementation with an end-to-end use-case evaluation of each module.

II. RELATED WORK

A. Hybrid Post-Quantum Key Exchange Mechanisms

Kwiatkowski et al. [1] combine X25519 ECDHE with ML-KEM-768 to define hybrid key-agreement mechanisms for TLS 1.3, directly supporting the hybrid key-exchange approach adopted in this work. Prikutse et al. [2] propose a commitment-based hybrid post-quantum cryptographic model for multi-file cloud storage using AES-256-GCM, X25519, and ML-KEM-768, closely relating to the hybrid AES-256-GCM, X25519, and ML-KEM-768 construction used here. Olushola and Meenakshi [3] design an authenticated post-quantum session protocol using ML-KEM, ML-DSA, HKDF, and AES-256-GCM,

supporting the choice of key-derivation and authenticated-encryption primitives used in this system. Shaw [4] implements and evaluates a hybrid-by-default Python cryptography library combining X25519 with ML-KEM-768, demonstrating that such hybrid constructions are practical to implement in production software. Shakib et al. [8] present AmphiKey, a dual-mode authenticated key-encapsulation protocol combining ML-KEM-768 and X25519 with forward secrecy, strongly supporting the hybrid key-establishment design used in this project. A further study [9] combines ECC, Kyber-768, and AES-256-GCM for secure digital communication, reinforcing the pattern of pairing a lattice-based KEM with a classical curve and a symmetric AEAD cipher, while [16] surveys practical methods for combining classical, quantum, and post-quantum key-exchange mechanisms and directly supports combining classical X25519 with ML-KEM as done in this work.

B. Performance and Deployment Studies

Dubey and Varshney [5] measure the post-quantum readiness of the Internet and report increasing real-world adoption of ML-KEM-768 with X25519. A benchmarking study [6] compares post-quantum and classical cryptography on performance and communication overhead at the micro and transport layers, while another [7] evaluates post-quantum cryptography performance specifically for MQTT over TLS on constrained, ARM-based IoT devices — both directly relevant to deploying ML-KEM-768 on resource-constrained Android hardware. A further wireless-focused study [10] evaluates post-quantum algorithms combined with symmetric encryption over wireless networks, closely matching the secure, nearby, wireless file-transfer scenario addressed in this paper. Chang [11] demonstrates the practical application of ML-KEM to key exchange and identity protection in 5G networks, and a broader performance study [18] measures the effect of post-quantum cryptography on TLS 1.3 latency, bandwidth, and network performance. Tzinos et al. [19] evaluate Kyber and hybrid post-quantum schemes under varying network conditions, and B rstinghaus-Steinbach et al. [20] integrate and evaluate Kyber and SPHINCS+ with mbed TLS on resource-constrained embedded systems, together demonstrating that hybrid post-quantum cryptography is practical even on constrained platforms such as the Android devices targeted by this work.

C. Formal Verification and Side-Channel Security

Bhargavan et al. [14] formally verify the PQXDH post-quantum key-agreement protocol used in end-to-end secure messaging, supporting the correctness of hybrid PQ session establishment adopted in this design. A related study [15] applies formally sound CryptoVerif analysis to hybrid TLS and SSH key exchanges, underscoring the importance of formally validating hybrid key-exchange security before deployment. Chen et al. [17] analyze side-channel vulnerabilities introduced by data memory-dependent prefetchers that can break constant-time cryptographic implementations, highlighting the need for careful, side-

channel-aware implementation of the cryptographic primitives used in this system.

D. Surveys and Domain-Specific Applications

Chhetri et al. [12] survey post-quantum algorithms, hybrid cryptography, migration strategies, and deployment challenges, providing background for the cryptographic choices made in this paper. A further survey [13] proposes enhanced network-security protocols for the quantum era, combining classical and post-quantum cryptography together with quantum key distribution, supporting the broader case for hybrid cryptography during the transition to quantum-safe systems. Collectively, this body of work strongly supports the central design decision of this paper — combining ML-KEM-768 with X25519 — while also revealing a common gap: limited exploration of combining hybrid PQ key exchange with a fully offline, serverless, peer-to-peer transport on commodity mobile devices. This is the gap the present work addresses.

III. THREAT MODEL AND DESIGN GOALS

The system is designed against an adversary who can observe and inject packets on the local wireless medium, and who may attempt to impersonate a nearby device during discovery. The adversary is not assumed to have compromised either endpoint device itself, nor is the adversary assumed to have a large-scale quantum computer today; however, the design assumes such an adversary may exist in the future and may record ciphertext now for decryption later (a "harvest-now, decrypt-later" attacker), which is the primary motivation for using ML-KEM-768 alongside X25519 rather than X25519 alone.

Given this adversary, the system pursues four concrete design goals: (i) authenticity — a device must not be able to complete key exchange with a peer unless a human has verified an out-of-band pairing value, preventing silent man-in-the-middle substitution during discovery; (ii) confidentiality — file contents must remain unreadable to any party other than the sender and the intended recipient; (iii) forward security at the session level — ephemeral keys and derived session keys are discarded immediately after each transfer so that compromise of a later session does not expose earlier transfers; and (iv) minimal trust in infrastructure — the system must function with no server or internet connectivity at all, so that no third-party infrastructure needs to be trusted, consistent with the zero-trust posture argued for in [2] and [9].

IV. PROPOSED SYSTEM ARCHITECTURE

A. System Overview

The proposed system operates in an offline peer-to-peer mode, illustrated in Fig. 1. It chains together peer discovery, out-of-band pairing, a hybrid key exchange, key derivation, and authenticated encryption to achieve a fully serverless, air-gapped transfer.

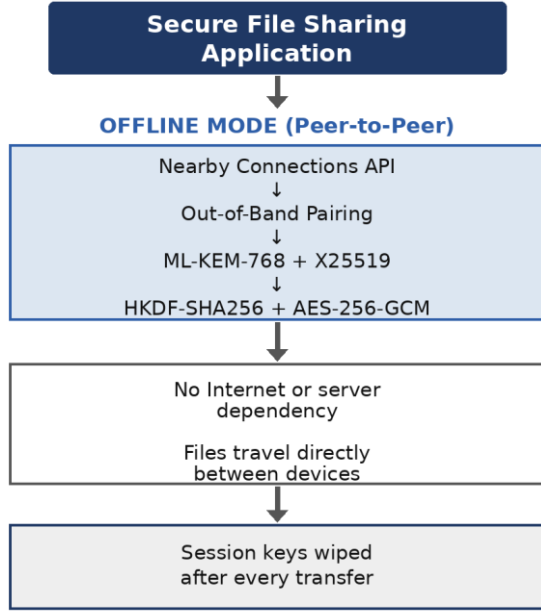


Fig. 1. High-level overview of the proposed system.

B. Offline (Peer-to-Peer) Architecture

For offline transfer, two peer devices first discover each other using the Nearby Connections API and verify a shared out-of-band pairing code before proceeding. The devices then exchange public keys and perform a hybrid key exchange using X25519 combined with ML-KEM-768, after which HKDF-SHA256 combines both resulting secrets into a single session key. The file is then transferred using AES-256-GCM authenticated encryption, and finally all ephemeral keys and the session key are wiped from memory. This sequence is summarized in Fig. 2 and formalized in Algorithm 1.

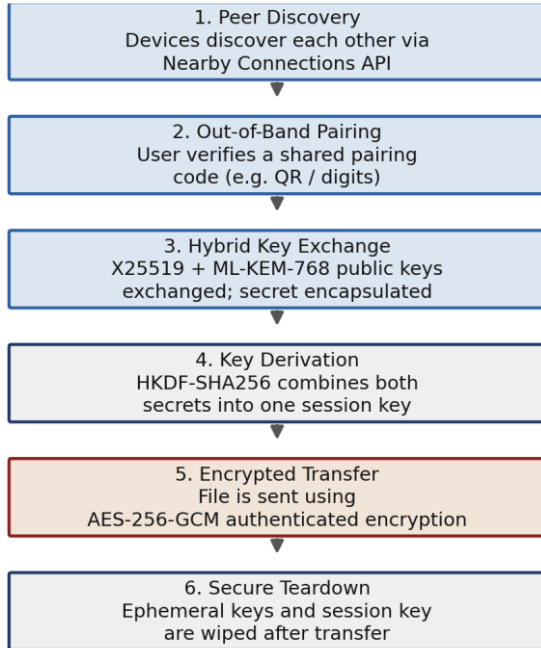


Fig. 2. Offline peer-to-peer key-exchange and transfer sequence.

Algorithm 1: Offline Hybrid Key Exchange & Transfer

```

Input: Device A (sender), Device B (receiver)
1: A, B discover each other via Nearby
Connections
2: A, B display/scan pairing code P
3: if P_A ≠ P_B then abort
4: A generates (skA, pkA) on X25519
5: A generates ML-KEM-768 keypair (ekA, dkA)
6: B generates (skB, pkB) on X25519
7: A sends (pkA, ekA) to B
8: B computes (ss_kem, ct) ← Encaps(ekA)
9: B computes ss_ecdh ← X25519(skB, pkA)
10: B sends (ct, pkB) to A
11: A computes ss_kem ← Decaps(dkA, ct)
12: A computes ss_ecdh ← X25519(skA, pkB)
13: K ← HKDF-SHA256(ss_kem || ss_ecdh)
14: C ← AES-256-GCM.Encrypt(K, File)
15: Transfer C over the established channel
16: Erase skA, skB, dkA, ss_kem, ss_ecdh, K
  
```

C. Tools and Technologies

The application is implemented on Android using Kotlin with Jetpack Compose and Material 3 for the UI. Cryptographic primitives (ML-KEM-768, X25519, HKDF-SHA256, AES-256-GCM) are implemented using the Bouncy Castle library. Offline peer-to-peer connectivity uses the Nearby Connections API.

V. IMPLEMENTATION STATUS

Table I summarizes the current implementation status of each module in the end-to-end use case, for the offline pathway, at the time of this evaluation.

Step	Module	Status
Device discovery	Nearby Connections API	Live
Out-of-band pairing	Pairing (magic number)	Live
Key exchange	Quantum-secure KEX	Live (in-memory)
Derive session key	HKDF-SHA256	Live
Send encrypted file	AES-256-GCM transfer	Live (I/O stubbed)
Receive & decrypt	Client-side decryption	Live

TABLE I. END-TO-END USE CASE STATUS

VI. COMMUNICATION AND STORAGE OVERHEAD

A practical concern with any post-quantum key-encapsulation mechanism is the increase in key and ciphertext sizes relative to classical elliptic-curve schemes. Table III lists the approximate public-key, ciphertext, and shared-secret sizes for the primitives used in this system, alongside a classical RSA-3072 baseline for comparison. Although ML-KEM-768 public keys and ciphertexts are roughly 30–35 times larger than their X25519 counterparts, the resulting shared secret remains a compact 32 bytes, and the absolute overhead (on the order of a few kilobytes per session) is negligible relative to the size of most shared files and well within the bandwidth budget of the Nearby Connections API.

Primitive	Public Key	Ciphertext
X25519	32 bytes	32 bytes
ML-KEM-768	1184 bytes	1088 bytes
RSA-3072 (baseline)	384 bytes	384 bytes
AES-256-GCM	32-byte key	+16-byte tag

TABLE III. APPROXIMATE CRYPTOGRAPHIC PARAMETER SIZES

This overhead is paid only once per session, since the key exchange is performed a single time between the two devices and the file itself is then encrypted with a compact 32-byte symmetric key, keeping the scheme efficient regardless of file size.

VII. RESULTS AND DISCUSSION

At the current stage of implementation, offline peer-to-peer device discovery and connection using the Nearby Connections API has been successfully established without requiring internet connectivity, along with user-verified device authentication using authentication digits prior to accepting a connection. The ML-KEM-768 + X25519 hybrid key-exchange workflow and HKDF-SHA256 session-key derivation have both been implemented and verified, as has AES-256-GCM encryption and decryption with authentication-tag verification for file integrity. Encrypted file transfer and authenticated decryption between connected devices have been verified, confirming that plaintext file data is never transmitted during the process. Session-specific ephemeral keys prevent reuse of long-term secrets across transfers, and secure session handling removes ephemeral and session keys once a transfer completes.

These results validate the core premise of this work: a hybrid post-quantum key-exchange scheme can be practically implemented on a resource-constrained Android device without prohibitive overhead — consistent with the constrained-device performance findings of [7], [10], and [20] — and performing key exchange and encryption directly between the two devices removes the need to trust any server infrastructure. The main outstanding item is the direct integration of the negotiated session key with the live Nearby Connections data channel, since key exchange has so far been verified in-memory rather than over the network transport; this integration, along with QR-based pairing and performance benchmarking, is planned for future work.

VIII. COMPARISON WITH EXISTING SYSTEMS

Table II situates the proposed system relative to common existing approaches to file sharing and secure key exchange, highlighting the specific weaknesses that motivated the present design.

Existing Approach	Key Limitation vs. Proposed System
Cloud drives (Drive, Dropbox)	Server stores/accesses plaintext files; ours uses no server, files go directly device-to-device.
Closed cryptosystems (e.g. DES)	Undisclosed constants enable backdoors; we use openly reviewed PQ algorithms.

Existing Approach	Key Limitation vs. Proposed System
Traditional nearby file transfer	No user-verified peer auth. or PQ key exchange; we add verified pairing + ML-KEM-768.
RSA / ECC key exchange	Breakable by a sufficiently capable quantum computer; no known attack on our hybrid scheme.

TABLE II. COMPARISON WITH EXISTING SYSTEMS

Across all four comparisons, the recurring theme is a reduction in required trust — in cloud providers, in closed cryptographic designs, in unauthenticated nearby-transfer protocols, and in the long-term security of purely classical key exchange — which together motivate the hybrid, minimal-trust architecture proposed in this paper.

IX. CONCLUSION AND FUTURE WORK

This paper presented a secure file-sharing architecture supporting offline peer-to-peer file transfer. The system combines the Nearby Connections API, user-verified pairing, a hybrid ML-KEM-768 + X25519 key exchange, HKDF-SHA256, and AES-256-GCM to provide secure local file transfer without internet or server dependency. A threat model grounded in a "harvest-now, decrypt-later" adversary, together with a survey of twenty recent works on post-quantum cryptography, was used to justify each design decision. Overall, the proposed architecture demonstrates the feasibility of combining offline peer-to-peer communication, post-quantum cryptography, and authenticated encryption into a single, lightweight, secure file-sharing system.

Future work includes completing end-to-end integration of the key exchange with the live Nearby Connections data channel, adding QR-based pairing as an alternative to manually comparing authentication digits, incorporating ML-DSA-based digital signatures for stronger device-identity verification, introducing chunked encryption for large-file support, extending offline sharing to controlled multi-device sessions, adding an online, server-assisted multi-recipient sharing mode, benchmarking performance across different Android devices, adding persistent trusted-device identity management, and conducting formal security analysis and penetration testing of the complete protocol, guided by the side-channel and formal-verification concerns raised in [14], [15], and [17].

ACKNOWLEDGMENT

The authors would like to thank the Department of Computer Science and Engineering, Rajalakshmi Engineering College, Chennai, for providing the infrastructure and support necessary to carry out this work, and are especially grateful to their project supervisor, Mr. Bhuvaneshwaran B, Assistant Professor (SG), for his continued guidance, technical feedback, and encouragement throughout the design and implementation of this system.

REFERENCES

- [1] K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan, and D. Stebila, "Post-Quantum Traditional (PQ/T) Hybrid Key

- Agreement Mechanisms for TLS 1.3,” RFC 10024, Internet Engineering Task Force (IETF), Aug. 2026, doi: 10.17487/RFC10024.
- [2] L. F. Prikutse, R. E. Turkson, A.-S. Yussiff, A.-L. Yussiff, and M. G. Asante-Mensah, “A Commitment-Based Hybrid Post-Quantum Cryptographic Model for Multi-File Cloud Storage,” arXiv preprint arXiv:2608.13138, 2026.
- [3] A. O. Olushola and S. P. Meenakshi, “Design and implementation of an authenticated post-quantum session protocol using ML-KEM (Kyber), ML-DSA (Dilithium), and AES-256-GCM,” *Front. Phys.*, vol. 13, Art. no. 1723966, 2026, doi: 10.3389/fphy.2025.1723966.
- [4] A. Shaw, “quantum-safe: Bridging the Post-Quantum Production Gap with a Hybrid-by-Default Python Cryptography Library,” arXiv preprint arXiv:2605.17061, 2026.
- [5] V. M. Dubey and G. Varshney, “Measurement Study of Post-Quantum Readiness of Internet: 2026,” arXiv preprint arXiv:2606.16473, 2026.
- [6] E. D. Demir, I. Gorgulu, and M. C. Onbaşlı, “Micro and Transport Layer Security Benchmarking of Post-Quantum Versus Classical Cryptography,” *IEEE Access*, vol. 14, pp. 56099–56111, 2026, doi: 10.1109/ACCESS.2026.3682760.
- [7] E. C. Santos, T. A. O. D. Cordeiro, and H. de Oliveira Silva, “Performance Evaluation of Post-Quantum Cryptography in IoT: A Case Study on MQTT Over TLS Under Network Constraints,” in *Proc. 14th Int. Symp. Digital Forensics and Security (ISDFS)*, 2026, pp. 1–6, doi: 10.1109/ISDFS69419.2026.11458974.
- [8] K. H. Shakib, M. A. Hafeez, and A. Munir, “AmphiKey: A Dual-Mode Secure Authenticated Key Encapsulation Protocol for Smart Grid,” arXiv preprint arXiv:2509.01701, 2025.
- [9] W. R. Maya, T. Y. Arif, H. Riza, and R. Adriman, “Optimized and Secure Hybrid Key Exchange Model for Post-Quantum Digital Communications,” in *Proc. 3rd Int. Conf. Computer System, Information Technology, and Electrical Engineering (COSITE)*, 2025, doi: 10.1109/COSITE68330.2025.11414259.
- [10] B. Ojetunde, T. Kurihara, K. Yano, T. Sakano, et al., “Performance Evaluation of Post-Quantum Cryptography Algorithms for Secure Communication in Wireless Networks,” in *Proc. IEEE 22nd Consumer Communications & Networking Conf. (CCNC)*, 2025, doi: 10.1109/CCNC54725.2025.10975932.
- [11] Q. Khan, S. Purification, and S.-Y. Chang, “Post-Quantum Key Exchange and ID Encryption Analyses for 5G Mobile Networking,” in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2025, doi: 10.1109/NOMS57970.2025.11073683.
- [12] G. Chhetri, S. Somvanshi, P. Hebli, S. Brotee, and S. Das, “Post-Quantum Cryptography and Quantum-Safe Security: A Comprehensive Survey,” arXiv preprint arXiv:2510.10436, 2025.
- [13] C. Rubio García, A. Cano Aguilera, C. I. Stan, J. J. Vegas Olmos, S. Rommel, and I. Tafur Monroy, “Enhanced Network Security Protocols for the Quantum Era: Combining Classical and Post-Quantum Cryptography, and Quantum Key Distribution,” *IEEE J. Sel. Areas Commun.*, vol. 43, no. 8, pp. 2765–2781, 2025, doi: 10.1109/JSAC.2025.3568011.
- [14] K. Bhargavan, C. Jacomme, F. Kiefer, and R. Schmidt, “Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol for End-to-End Secure Messaging,” in *Proc. 33rd USENIX Security Symp.*, pp. 469–486, 2024.
- [15] B. Blanchet and C. Jacomme, “Post-Quantum Sound CryptoVerif and Verification of Hybrid TLS and SSH Key-Exchanges,” in *Proc. IEEE 37th Computer Security Foundations Symp. (CSF)*, pp. 543–556, 2024, doi: 10.1109/CSF61375.2024.00050.
- [16] S. Ricci, P. Dobiáš, L. Malina, J. Hajný, and P. Jedlička, “Hybrid Keys in Practice: Combining Classical, Quantum and Post-Quantum Cryptography,” *IEEE Access*, vol. 12, pp. 23206–23219, 2024, doi: 10.1109/ACCESS.2024.3364520.
- [17] B. Chen, Y. Wang, P. Shome, C. Fletcher, D. Kohlbrenner, R. Paccagnella, and D. Genkin, “GoFetch: Breaking Constant-Time Cryptographic Implementations Using Data Memory-Dependent Prefetchers,” in *Proc. 33rd USENIX Security Symp.*, pp. 1117–1134, 2024.
- [18] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, and G. Carle, “The Performance of Post-Quantum TLS 1.3,” in *Proc. 19th ACM Int. Conf. on Emerging Networking Experiments and Technologies (CoNEXT)*, 2023, doi: 10.1145/3624354.3630585.
- [19] I. Tzinos, K. Limniotis, and N. Kolokotronis, “Evaluating the Performance of Post-Quantum Secure Algorithms in the TLS Protocol,” *J. Surveill. Secur. Saf.*, vol. 3, pp. 101–127, 2022, doi: 10.20517/jsss.2022.15.
- [20] K. Bürstinghaus-Steinbach, C. Krauß, R. Niederhagen, and M. Schneider, “Post-Quantum TLS on Embedded Systems: Integrating and Evaluating Kyber and SPHINCS+ with mbed TLS,” in *Proc. 15th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, pp. 841–852, 2020, doi: 10.1145/3320269.3384725.